

# Bài giảng: Views trong MySQL

CREATE VIEW, View Processing, Updatable Views, WITH CHECK OPTION,  
Show/Rename/Drop Views

CSDL / MySQL

Ngày 25 tháng 6 năm 2026

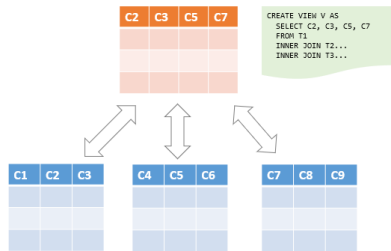
Nguồn tham khảo chính: MySQLTutorial.org - MySQL Views.

Sau bài này, sinh viên có thể:

- Giải thích được view là gì và vì sao view được gọi là bảng ảo.
- Tạo view bằng `CREATE VIEW` và truy vấn view như một bảng.
- Phân biệt các thuật toán xử lý view: `MERGE`, `TEMPTABLE`, `UNDEFINED`.
- Xác định điều kiện để một view có thể cập nhật được.
- Dùng `WITH CHECK OPTION`, `LOCAL`, `CASCADE` để kiểm soát tính nhất quán.
- Quản lý view: `show`, `show create`, `rename`, `drop`.

# View là gì?

- View là một **truy vấn được đặt tên** và lưu trong database catalog.
- View có thể được truy vấn bằng SELECT giống như bảng.
- View thường **không lưu dữ liệu vật lý**; khi truy vấn, MySQL thực thi câu SELECT bên trong view.
- Vì vậy, view thường được gọi là **virtual table**.



# Ví dụ động cơ: lặp lại truy vấn JOIN

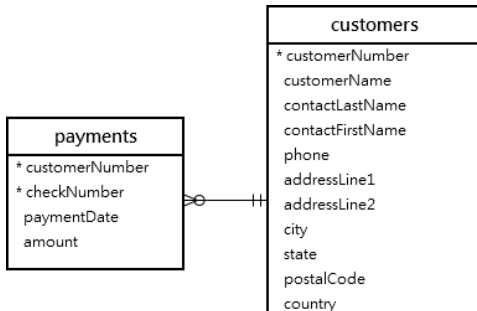
## SELECT

```
customerName,  
checkNumber,  
paymentDate,  
amount
```

FROM customers

INNER JOIN payments USING (customerNumber);

- Nếu truy vấn này được dùng nhiều lần, việc viết lại sẽ dài và dễ sai.
- Ta có thể lưu truy vấn này thành một view để dùng lại.



# Tạo view đầu tiên

```
CREATE VIEW customerPayments AS
SELECT
    customerName,
    checkNumber,
    paymentDate,
    amount
FROM customers
INNER JOIN payments USING (customerNumber);
```

Sau đó truy vấn ngắn gọn hơn:

```
SELECT * FROM customerPayments;
```

Ý tưởng chính: view giúp đóng gói truy vấn dài thành một tên dễ dùng.

## Lợi ích của view

- ❶ **Đơn giản hoá truy vấn phức tạp:** dùng `SELECT * FROM view_name` thay cho truy vấn dài.
- ❷ **Nhất quán logic nghiệp vụ:** công thức/tính toán được định nghĩa một lần.
- ❸ **Tăng lớp bảo mật:** chỉ expose một số cột/dòng cần thiết.
- ❹ **Tương thích ngược:** ứng dụng cũ có thể truy cập view như bảng cũ sau khi tái cấu trúc CSDL.

|   | orderNumber | total    |
|---|-------------|----------|
| ▶ | 10165       | 67392.85 |
|   | 10287       | 61402.00 |
|   | 10310       | 61234.67 |
|   | 10212       | 59830.55 |
|   | 10207       | 59265.14 |
|   | 10127       | 58841.35 |
|   | 10204       | 58793.53 |
|   | 10126       | 57131.92 |
|   | 10222       | 56822.65 |
|   | 10142       | 56052.56 |

# Cú pháp CREATE VIEW

```
CREATE [OR REPLACE] VIEW [db_name.]view_name [(column_list)]  
AS  
    select_statement;
```

- view\_name: tên view, không được trùng tên bảng/view trong cùng database.
- OR REPLACE: thay thế view nếu view đã tồn tại.
- column\_list: đặt tên cột cho view, có thể bỏ qua nếu lấy từ SELECT.
- select\_statement: câu truy vấn định nghĩa dữ liệu mà view hiển thị.

## Ví dụ 1: View tính tổng doanh số mỗi đơn hàng

```
CREATE VIEW salePerOrder AS
SELECT
    orderNumber,
    SUM(quantityOrdered * priceEach) AS total
FROM orderDetails
GROUP BY orderNumber
ORDER BY total DESC;
```

```
SELECT * FROM salePerOrder;
```

|   | customerName               | checkNumber | paymentDate | amount   |
|---|----------------------------|-------------|-------------|----------|
| ► | Atelier graphique          | HQ336336    | 2004-10-19  | 6066.78  |
|   | Atelier graphique          | JM555205    | 2003-06-05  | 14571.44 |
|   | Atelier graphique          | OM314933    | 2004-12-18  | 1676.14  |
|   | Signal Gift Stores         | BO864823    | 2004-12-17  | 14191.12 |
|   | Signal Gift Stores         | HQ55022     | 2003-06-06  | 32641.98 |
|   | Signal Gift Stores         | ND748579    | 2004-08-20  | 33347.88 |
|   | Australian Collectors, Co. | GG31455     | 2003-05-20  | 45864.03 |
|   | Australian Collectors, Co. | MA765515    | 2004-12-15  | 82261.22 |
|   | Australian Collectors, Co. | NP603840    | 2003-05-31  | 7565.08  |
|   | Australian Collectors, Co. | NR27552     | 2004-03-10  | 44894.74 |



## Ví dụ 2: Tạo view dựa trên view khác

```
CREATE VIEW bigSalesOrder AS
SELECT
    orderNumber,
    ROUND(total, 2) AS total
FROM salePerOrder
WHERE total > 60000;
```

```
SELECT orderNumber, total
FROM bigSalesOrder;
```

View có thể được định nghĩa dựa trên bảng gốc hoặc dựa trên một view đã có.

## Ví dụ 3: View với JOIN nhiều bảng

```
CREATE OR REPLACE VIEW customerOrders AS
SELECT
    orderNumber,
    customerName,
    SUM(quantityOrdered * priceEach) AS total
FROM orderDetails
INNER JOIN orders USING (orderNumber)
INNER JOIN customers USING (customerNumber)
GROUP BY orderNumber;
```

|   | orderNumber | customerName                 | total    |
|---|-------------|------------------------------|----------|
| ► | 10165       | Dragon Souvenirs, Ltd.       | 67392.85 |
|   | 10287       | Vida Sport, Ltd              | 61402.00 |
|   | 10310       | Toms Spezialitäten, Ltd      | 61234.67 |
|   | 10212       | Euro+ Shopping Channel       | 59830.55 |
|   | 10207       | Diecast Collectables         | 59265.14 |
|   | 10127       | Muscle Machine Inc           | 58841.35 |
|   | 10204       | Muscle Machine Inc           | 58793.53 |
|   | 10126       | Corrida Auto Replicas, Ltd   | 57131.92 |
|   | 10222       | Collectable Mini Designs Co. | 56822.65 |
|   | 10142       | Mini Gifts Distributors Ltd  | 56052.56 |

## Ví dụ 4: View với subquery

```
CREATE VIEW aboveAvgProducts AS
SELECT
    productCode,
    productName,
    buyPrice
FROM products
WHERE buyPrice > (
    SELECT AVG(buyPrice)
    FROM products
)
ORDER BY buyPrice DESC;
```

- View này hiển thị các sản phẩm có giá mua lớn hơn giá mua trung bình.
- Người dùng view không cần biết subquery bên trong được viết như thế nào.

## Ví dụ 5: Đặt tên cột tường minh cho view

```
CREATE VIEW customerOrderStats (  
    customerName,  
    orderCount  
) AS  
SELECT  
    customerName,  
    COUNT(orderNumber)  
FROM customers  
INNER JOIN orders USING (customerNumber)  
GROUP BY customerName;
```

- Dùng `column_list` khi muốn tên cột của view rõ ràng hơn.
- Hữu ích khi biểu thức trong `SELECT` không có alias đẹp.

# Thuật toán xử lý view trong MySQL

MySQL có thể xử lý view theo 3 lựa chọn:

- MERGE
- TEMPTABLE
- UNDEFINED

```
CREATE [OR REPLACE]
ALGORITHM = {MERGE | TEMPTABLE | UNDEFINED}
VIEW view_name [(column_list)] AS
    select_statement;
```

Thuật toán ảnh hưởng đến cách MySQL biến truy vấn trên view thành truy vấn thực thi.

# MERGE algorithm

- MySQL kết hợp truy vấn bên ngoài với câu SELECT định nghĩa view.
- Kết quả là một truy vấn duy nhất trên bảng gốc.
- Thường hiệu quả hơn vì có thể tận dụng điều kiện lọc trực tiếp trên bảng gốc.

|   | orderNumber | total    |
|---|-------------|----------|
| ▶ | 10165       | 67392.85 |
|   | 10287       | 61402.00 |
|   | 10310       | 61234.67 |

# Ví dụ MERGE

```
CREATE ALGORITHM = MERGE VIEW contactPersons AS
SELECT
    customerName,
    contactFirstName AS firstName,
    contactLastName AS lastName,
    phone
FROM customers;
```

Truy vấn trên view:

```
SELECT * FROM contactPersons
WHERE customerName LIKE '%Co%';
```

Có thể được hiểu như truy vấn trực tiếp trên customers với điều kiện WHERE tương ứng.

# TEMPTABLE và UNDEFINED

**Thuật toán** Ý nghĩa

**TEMPTABLE** MySQL tạo bảng tạm chứa kết quả của câu SELECT trong view, sau đó truy vấn bảng tạm. Thường kém hiệu quả hơn MERGE; view dạng này không updatable.

**UNDEFINED** MySQL tự chọn thuật toán. Đây là mặc định nếu không chỉ định ALGORITHM. MySQL thường ưu tiên MERGE nếu có thể.

|   | productCode | productName                    | buyPrice |
|---|-------------|--------------------------------|----------|
| ▶ | S10_4962    | 1962 LanciaA Delta 16V         | 103.42   |
|   | S18_2238    | 1998 Chrysler Plymouth Prowler | 101.51   |
|   | S10_1949    | 1952 Alpine Renault 1300       | 98.58    |
|   | S24_3856    | 1956 Porsche 356A Coupe        | 98.3     |
|   | S12_1108    | 2001 Ferrari Enzo              | 95.59    |
|   | S12_1099    | 1968 Ford Mustang              | 95.34    |



# Updatable view là gì?

- Một số view không chỉ dùng để SELECT, mà còn có thể INSERT, UPDATE, DELETE.
- Khi cập nhật qua view, dữ liệu thật trong bảng gốc được thay đổi.
- Điều này tiện lợi nhưng cần kiểm soát kỹ để tránh cập nhật ngoài phạm vi view.

| <b>customers</b>                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| * customerNumber<br>customerName<br>contactLastName<br>contactFirstName<br>phone<br>addressLine1<br>addressLine2<br>city<br>state<br>postalCode<br>country<br>salesRepEmployeeNumber<br>creditLimit |

## Khi nào view thường không cập nhật được?

Một view thường **không updatable** nếu SELECT định nghĩa view chứa:

- Hàm tổng hợp: SUM, COUNT, AVG, MIN, MAX, ...
- DISTINCT, GROUP BY, HAVING.
- UNION hoặc UNION ALL.
- Outer join / left join.
- Subquery phụ thuộc vào bảng trong FROM.
- View dùng thuật toán TEMPTABLE.

# Ví dụ updatable view

```
CREATE VIEW officeInfo AS  
SELECT officeCode, phone, city  
FROM offices;
```

Cập nhật qua view:

```
UPDATE officeInfo  
SET phone = '+33 14 723 5555'  
WHERE officeCode = 4;
```

Nếu view đủ điều kiện updatable, lệnh UPDATE trên view sẽ cập nhật bảng offices.

# Kiểm tra view có updatable không

Có thể kiểm tra metadata trong `information_schema.views`:

```
SELECT
    table_name,
    is_updatable
FROM information_schema.views
WHERE table_schema = 'classicmodels';
```

|   | officeCode | phone            | city          |
|---|------------|------------------|---------------|
| ▶ | 1          | +1 650 219 4782  | San Francisco |
|   | 2          | +1 215 837 0825  | Boston        |
|   | 3          | +1 212 555 3000  | NYC           |
|   | 4          | +33 14 723 4404  | Paris         |
|   | 5          | +81 33 224 5000  | Tokyo         |
|   | 6          | +61 2 9264 2451  | Sydney        |
|   | 7          | +44 20 7877 2041 | London        |

# Xóa dữ liệu qua view

Tạo view lọc các mặt hàng đắt tiền:

```
CREATE VIEW LuxuryItems AS  
SELECT *  
FROM items  
WHERE price > 700;
```

Xóa qua view:

```
DELETE FROM LuxuryItems  
WHERE id = 3;
```

- Nếu view updatable, dòng tương ứng trong bảng items cũng bị xóa.
- Cần phân quyền cẩn thận khi cho phép thao tác trên view.

## Vấn đề nhất quán của updatable view

Giả sử view chỉ hiển thị nhân viên loại 'Contractor'.

- Nếu view updatable, người dùng có thể chèn qua view.
- Không có kiểm tra bổ sung, họ có thể chèn một nhân viên 'Full-time' qua view.
- Dòng mới được thêm vào bảng gốc nhưng không xuất hiện trong view.
- Đây là tình huống làm view trở nên khó hiểu và thiếu nhất quán.

|   | table_name       | is_updatable |
|---|------------------|--------------|
| ▶ | aboveavgproducts | YES          |
|   | customerorders   | NO           |
|   | officeinfo       | YES          |
|   | saleperorder     | NO           |

# WITH CHECK OPTION

```
CREATE OR REPLACE VIEW contractors AS
SELECT id, type, name
FROM employees
WHERE type = 'Contractor'
WITH CHECK OPTION;
```

- WITH CHECK OPTION yêu cầu dòng được INSERT/UPDATE qua view phải vẫn nhìn thấy được qua view.
- Nếu không thoả điều kiện WHERE của view, MySQL từ chối thao tác.

```
INSERT INTO contractors(name, type)
VALUES ('Brad Knox', 'Full-time');
-- ERROR: CHECK OPTION failed
```

## LOCAL và CASCADED trong WITH CHECK OPTION

Khi view được xây dựng chồng lên view khác, cần xác định phạm vi kiểm tra:

**Tuỳ chọn** Ý nghĩa

**LOCAL** Chỉ kiểm tra điều kiện của view hiện tại có khai báo WITH LOCAL CHECK OPTION.

**CASCADED** Kiểm tra điều kiện của view hiện tại và các view phụ thuộc bên dưới. Đây thường là lựa chọn an toàn hơn và là mặc định khi không ghi rõ.

|   | id | name   | price  |
|---|----|--------|--------|
| ▶ | 1  | Laptop | 700.56 |



# Ví dụ ý tưởng LOCAL/CASCADED

```
CREATE VIEW v_contractors AS  
SELECT * FROM employees  
WHERE type = 'Contractor'  
WITH CHECK OPTION;
```

```
CREATE VIEW v_hanoi_contractors AS  
SELECT * FROM v_contractors  
WHERE city = 'Ha Noi'  
WITH CASCADED CHECK OPTION;
```

- Chèn/cập nhật qua v\_hanoi\_contractors phải thỏa cả type = 'Contractor' và city = 'Ha Noi'.
- Nếu dùng LOCAL, cần đọc kỹ view nào được kiểm tra để tránh lọt dữ liệu không mong muốn.

# Show views trong database

Cách 1: dùng SHOW FULL TABLES:

```
SHOW FULL TABLES  
WHERE table_type = 'VIEW';
```

Tìm trong database cụ thể:

```
SHOW FULL TABLES IN classicmodels  
WHERE table_type = 'VIEW';
```

Tìm theo pattern:

```
SHOW FULL TABLES FROM classicmodels  
LIKE 'customer%';
```

# Show views bằng INFORMATION\_SCHEMA

```
SELECT table_name AS view_name
FROM information_schema.tables
WHERE table_type = 'VIEW'
      AND table_schema = 'classicmodels';
```

Tìm view có tên bắt đầu bằng customer:

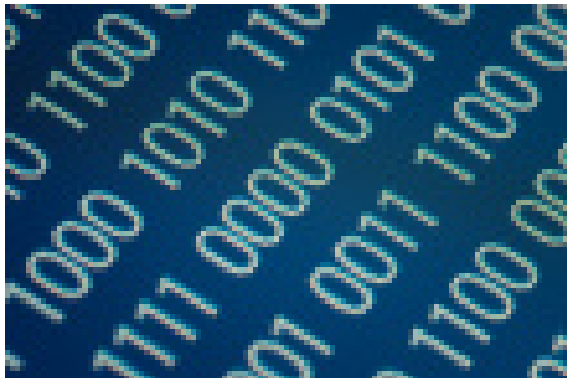
```
SELECT table_name AS view_name
FROM information_schema.tables
WHERE table_type = 'VIEW'
      AND table_schema = 'classicmodels'
      AND table_name LIKE 'customer%';
```

# SHOW CREATE VIEW

Dùng để xem lại câu lệnh tạo view:

```
SHOW CREATE VIEW customerPayments;
```

- Hữu ích khi cần kiểm tra định nghĩa view hiện tại.
- Dùng khi debug: view đang lọc điều kiện nào, join bảng nào, có WITH CHECK OPTION không.



# Rename views

MySQL có thể đổi tên view bằng RENAME TABLE:

```
RENAME TABLE old_view_name TO new_view_name;
```

Ví dụ:

```
RENAME TABLE customerPayments TO customerPaymentView;
```

- View và table dùng chung namespace, vì vậy tên mới không được trùng tên đối tượng đã có.
- Cần kiểm tra các câu SQL, stored procedure, app code đang phụ thuộc tên view cũ.

# Drop views

Xóa một view:

```
DROP VIEW view_name;
```

Xóa nhiều view:

```
DROP VIEW view_1, view_2;
```

Tránh lỗi nếu view không tồn tại:

```
DROP VIEW IF EXISTS view_name;
```

**DROP VIEW** chỉ xóa định nghĩa view, không xóa dữ liệu trong bảng gốc.

## Quy trình làm việc với view

- ❶ Xác định truy vấn cần dùng lại hoặc cần giới hạn dữ liệu.
- ❷ Tạo view bằng `CREATE VIEW`.
- ❸ Truy vấn thử bằng `SELECT`.
- ❹ Nếu cho phép cập nhật, kiểm tra view có updatable không.
- ❺ Với view lọc dữ liệu, cần nhắc thêm `WITH CHECK OPTION`.
- ❻ Quản lý view bằng `SHOW FULL TABLES`, `SHOW CREATE VIEW`, `RENAME TABLE`, `DROP VIEW`.

# Bài tập 1: Tạo view đơn giản

Cho bảng:

```
products(productCode, productName, productLine, buyPrice, MSRP)
```

Yêu cầu:

- 1 Tạo view productPriceView gồm productCode, productName, buyPrice, MSRP.
- 2 Truy vấn các sản phẩm có MSRP > 100 từ view.
- 3 Dùng SHOW CREATE VIEW để kiểm tra định nghĩa view.



## Bài tập 2: View tổng hợp không updatable

Cho bảng:

```
orderDetails(orderNumber, productCode, quantityOrdered, priceEach)
```

Yêu cầu:

- 1 Tạo view `orderTotalView` tính tổng tiền mỗi đơn hàng.
- 2 Kiểm tra view này có updatable không bằng `information_schema.views`.
- 3 Giải thích vì sao view này không nên/không thể cập nhật trực tiếp.

## Bài tập 3: WITH CHECK OPTION

Cho bảng:

```
employees(id, type, name, city)
```

Yêu cầu:

- 1 Tạo view contractors chỉ gồm nhân viên có type = 'Contractor'.
- 2 Thêm WITH CHECK OPTION.
- 3 Thử chèn một dòng type = 'Full-time' qua view và ghi nhận lỗi.
- 4 Giải thích vì sao lỗi này là đúng.

## Câu hỏi nhanh

- ❶ Vì sao view được gọi là virtual table?
- ❷ Khi nào nên dùng view thay vì viết lại truy vấn dài?
- ❸ MERGE khác TEMPTABLE như thế nào?
- ❹ Vì sao view có GROUP BY thường không updatable?
- ❺ WITH CHECK OPTION giải quyết vấn đề gì?
- ❻ Khi nào nên dùng SHOW CREATE VIEW?

- MySQLTutorial.org, *MySQL Views*.
- MySQLTutorial.org, *MySQL CREATE VIEW*.
- MySQLTutorial.org, *MySQL View Processing Algorithms*.
- MySQLTutorial.org, *Create MySQL Updatable Views*.
- MySQLTutorial.org, *MySQL WITH CHECK OPTION Clause*.
- MySQLTutorial.org, *MySQL Show View / SHOW CREATE VIEW / Rename View / DROP VIEW*.